```
import pandas as pd
import numpy as np

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

from google.colab import files
```

```
print("Upload TRAINING file (Book1.xlsx)")
uploaded_train = files.upload()

train_file_name = list(uploaded_train.keys())[0]
train_df = pd.read_excel(train_file_name)

train_df.head()
```

```
Upload TRAINING file (Book1.xlsx)
```

Choose Files Training Book1.xlsx

Training Book1.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 28688 bytes, last modified: 1/5/2026 - 100% done

```
Saving Training Book1.xlsx to Training Book1 (1).xlsx
```

	Message	Unnamed: 1	Unnamed: 2	Topic	Unnamed: 4	Unnamed: 5	Resolution Department
0	Request payoff scheduling and confirm lien rel...	NaN	NaN	Transaction Assistance	NaN	NaN	Payments Operations
1	Follow-up: Suspicious email about loan transfe...	NaN	NaN	Security Related	NaN	NaN	Fraud & Security
2	Second reminder: Suspicious email about loan t...	NaN	NaN	Security Related	NaN	NaN	Fraud & Security
3	Escalation request: Request payoff scheduling ...	NaN	NaN	Transaction Assistance	NaN	NaN	Payments Operations
4	Closure confirmation: Suspicious email about l...	NaN	NaN	Security Related	NaN	NaN	Fraud & Security

Next steps: Generate code with train_df New interactive sheet

```
print("Upload TESTING file (Book1_testing_messages.xlsx)")
uploaded_test = files.upload()

test_file_name = list(uploaded_test.keys())[0]
test_df = pd.read_excel(test_file_name)

test_df.head()
```

```
Upload TESTING file (Book1_testing_messages.xlsx)
```

Choose Files Testing Boo… topics.xlsx

Testing Book with topics.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 12694 bytes, last modified: 1/5/2026 - 100% done

```
Saving Testing Book with topics.xlsx to Testing Book with topics (1).xlsx
```

	Message	Unnamed: 1	Unnamed: 2	Topic
0	Request interest rate tiers and fee schedule f...	NaN	NaN	Information
1	Follow-up: Request interest rate tiers and fee...	NaN	NaN	Information
2	Second reminder: Request interest rate tiers a...	NaN	NaN	Information
3	Escalation request: ATM did not dispense cash ...	NaN	NaN	Complaints
4	Refund for canceled wire transfer fee requested.	NaN	NaN	Disputes

Next steps: Generate code with `test_df` New interactive sheet

```
X_train = train_df["Message"].astype(str)
y_train = train_df["Topic"].astype(str)
```

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-zA-Z0-9 ]", "", text)
    return text
```

```
vectorizer = TfidfVectorizer(
    stop_words="english",
    max_features=5000
)

X_train_tfidf = vectorizer.fit_transform(X_train)
```

```
model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced"
)

model.fit(X_train_tfidf, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(class_weight='balanced', n_estimators=200,
                       random_state=42)
```

```
X_test = test_df["Message"].astype(str)
X_test_tfidf = vectorizer.transform(X_test)

predicted_topics = model.predict(X_test_tfidf)

test_df["Predicted_Topic"] = predicted_topics
```

```
output_file = "classified_test_file.xlsx"
test_df.to_excel(output_file, index=False)

files.download(output_file)
```

```
from sklearn.metrics import accuracy_score

if "Topic" in test_df.columns:
    accuracy = accuracy_score(
        test_df["Topic"].astype(str),
        test_df["Predicted_Topic"].astype(str)
    )

    accuracy_percentage = accuracy * 100

    print(f"Accuracy Percentage: {accuracy_percentage:.2f}%")

else:
    print("No actual Topic column found. Accuracy cannot be calculated.")
```

```
Accuracy Percentage: 78.75%
```

```
correct = (test_df["Topic"] == test_df["Predicted_Topic"]).sum()
total = len(test_df)

print(f"Correct Predictions: {correct}/{total}")
```

```
Correct Predictions: 189/240
```

```
Start coding or generate with AI.
```